OPERATING SYSTEMS

Chapter 3: operating system structures

Prepared by: Yasmeen Alshraideh

ROOT TEAM
ROOT : WHERE SECURITY BEGINS

Slide 2 - Operating System Structures

- OS can be viewed by:
- Examining the services it provides
- Looking at the interface it makes available to users and programmers
- Disassembling the system into components and their interconnections

يمكن دراسة نظام التشغيل من ثلاث زوايا:

- الخدمات (Services) اللي بقدمها للمستخدمين وللبرامج.
- الواجهة (Interface) اللي بوفرها للمستخدمين وللمبرمجين.
- تفكيك النظام إلى مكوّنات (Components) وفهم كيف ترتبط ببعضها (Interconnections).

Slide 3 - System Components

- Large and complex system such as an OS can be created only by partitioning the system into smaller pieces.
- Each of these pieces should be a well-delineated portion of the system, with carefully defined inputs, outputs, and function.

نظام التشغيل نظام كبير ومعقد، فما بنقدر نبنيه إلا بتقسيمه إلى أجزاء أصغر.

كل جزء لازم يكون محدد بوضوح (Well-Delineated)، وله Inputs وOutputs ووظيفة (Function) معرّفة بدقة.

Slide 4 - System Components (cont.)

- Process Management
- Main-Memory Management
- File Management
- I/O System Management
- Secondary-Storage Management
- Networking
- Protection System
- Command-interpreter System

المكوّنات الثمانية لنظام التشغيل:

- Process Management: إدارة العمليات (البرامج قيد التنفيذ).
- Main-Memory Management: إدارة الذاكرة الرئيسية.
- File Management: إدارة الملفات.
- I/O System Management: إدارة نظام الإدخال والإخراج.
- Secondary-Storage Management: إدارة التخزين الثانوي (الأقراص).
- Networking: الشبكات.
- Protection System: نظام الحماية.
- Command-Interpreter System: مفسّر الأوامر

السلايدات القادمة بتشرح كل مكوّن على حدة، وشو الـ Activities اللي بيقوم فيها نظام التشغيل.

Slide 5 - Process Management

- A Process: a program in execution.
- A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task.
- These resources are either given to the process when it is created, or allocated to it while it is running.
- Some initialization data may be passed along when the process is created.

الـ Process هي برنامج قيد التنفيذ (Program in Execution).

لتنجز الـ Process مهمتها بتحتاج موارد مثل CPU Time وMemory وFiles وI/O Devices.

هاي الموارد ممكن تُعطى للـ Process لحظة إنشائها (When Created)، أو تُخصص لها أثناء التنفيذ (While Running).

وممكن تنمرّر للـ Process بيانات ابتدائية (Initialization Data) عند إنشائها، مثل اسم الملف اللي رح تشتغل عليه.

Slide 6 - Process Management (cont.)

- A Program by itself is not a process.
- A program is passive entity, such as the contents of a file stored on a disk.
- A process is an active entity, with a program counter specifying the next instruction to execute.
- The execution of a process must progress in a sequential fashion.

البرنامج (Program) لحاله مش Process.

الـ Program كيان ساكن (Passive Entity)، مثل محتوى ملف مخزّن على القرص.

الـ Process كيان نشط (Active Entity)، وعنده Program Counter بحدد التعليمة التالية اللي رح تنفّذ.

تنفيذ الـ Process لازم يتقدم بشكل تسلسلي (Sequential Fashion): تعليمة بعد تعليمة.

Slide 7 - Process Management (cont.)

- At any point of time, at most one instruction is executed on behalf of the process.
- If two processes are associated with same program, they are nevertheless considered two separate execution sequences.
- A process is the unit of work in a system.
- A system consists of a collection of processes, some of which are OS processes and the rest of which are user processes.

في أي لحظة، تنفّذ تعليمة واحدة على الأكثر لصالح الـ Process الواحدة.

لو في two processes مرتبطين بنفس الـ Program، بنعتبرهم مع هيك Execution Sequences منفصلين. مثال: تشغيل نسختين من نفس البرنامج يعطي two processes مختلفين.

الـ Process هي وحدة العمل (Unit of Work) بالنظام.

النظام عبارة عن مجموعة Processes: بعضها OS Processes(تابعة لنظام التشغيل) والباقي User Processes(تابعة للمستخدمين).

Slide 8 - Process Management (OS Activities)

- OS activities:
- The creation and deletion of user and system processes
- The suspension and resumption of processes
- The provision of mechanisms for process synchronization
- The provision of mechanisms for process communication
- The provision of mechanisms for deadlock handling

مهام نظام التشغيل في إدارة العمليات:

- إنشاء وحذف الـ Processes(للمستخدم وللنظام): Creation and Deletion.
- إيقاف Process مؤقتًا واستئنافها لاحقًا: Suspension and Resumption.
- توفير آليات للـ Synchronization: تنظيم عمل Processes تشترك بنفس البيانات.
- توفير آليات للـ Communication: تبادل المعلومات بين الـ Processes.
- توفير آليات للتعامل مع الـ Deadlock: حالة تتوقف فيها Processes لأن كل وحدة بتستنى الثانية.

Slide 9 - Main-Memory Management

- Main memory is central to the operation of a modern computer system.
- Main memory is a repository of quickly accessible data shared by the CPU and the I/O devices.
- The CPU reads instructions from main memory during instruction-fetch cycle, and both reads and writes data from main memory during the data-fetch cycle.

الـ Main Memory محور عمل الحاسوب الحديث.

هي مخزن بيانات سريعة الوصول (Quickly Accessible Data) يشترك فيها الـ CPU وأجهزة الـ I/O.

الـ CPU بقرأ التعليمات من الذاكرة أثناء Instruction-Fetch Cycle، وبقرأ ويكتب البيانات أثناء Data-Fetch Cycle.

Slide 10 - Main-Memory Management (cont.)

- The I/O operations implemented via DMA also read and write data in main memory.
- Main memory is generally the only large storage device that the CPU is able to address and access directly.
- There are many different memory-management schemes.
- Selection of memory management scheme for a specific system depends on many factors – especially on the hardware design of the system. Each algorithm requires its own hardware support.

عمليات الـ I/O اللي بتتم عبر DMA بتقرأ وتكتب بيانات بالـ Main Memory كمان.

الـ Main Memory عادةً هي الوسيط التخزيني الكبير الوحيد اللي الـ CPU بقدر يعنونه ويوصله مباشرة.

في مخططات إدارة ذاكرة (Memory-Management Schemes) مختلفة كثيرة.

اختيار المخطط المناسب لنظام معين بعتمد على عوامل كثيرة، وأهمها تصميم الـ Hardware، لأن كل Algorithm بحتاج دعم Hardware خاص فيه.

Slide 11 - Main-Memory Management (OS Activities)

- OS Activities:
- Keep track of which parts of memory are currently being used and by whom.
- Decide which processes are to be loaded into memory when memory space becomes available.
- Allocate and deallocate memory space as needed.

مهام نظام التشغيل في إدارة الذاكرة:

- تتبّع أي أجزاء من الذاكرة مستخدمة حاليًا ومن قبل مين .(Keep Track)
- تحديد أي Processes رح تنحمّل للذاكرة لما تتوفر مساحة فاضية.
- تخصيص مساحة الذاكرة وتحريرها عند الحاجة .(Allocate and Deallocate)

Slide 12 - File Management

- For convenient use of the computer system, the OS provides a uniform logical view of information storage. The OS abstracts from the physical properties of its storage devices to define a logical storage unit, the file.
- OS maps files onto physical media, and access these files via the storage devices.

عشان يكون استخدام الحاسوب مريح، بوفر نظام التشغيل رؤية منطقية موحّدة (Uniform Logical View) لتخزين المعلومات.

الـ OS بجرّد الخصائص الفيزيائية لأجهزة التخزين، وبعرّف وحدة تخزين منطقية اسمها الـ File.

بعدها بربط الملفات على الوسائط الفيزيائية، وبوصلها من خلال أجهزة التخزين.

Slide 13 - File Management (cont.)

- A file is a collection of related information defined by its creator.
- Commonly, files represent programs (both source and object forms) and data.
- Data files may be numeric, alphabetic, and alphanumeric.
- Files may be free-form, such as text files, or may be formatted rigidly.
- A file consists of a sequence of bits, bytes, lines, or records whose meaning is defined by their creators.

الـ File هو مجموعة معلومات مترابطة يحددها منشئه .(Creator)

عادةً الملفات بتمثل برامج (بصيغة Source وObject) وبيانات

ملفات البيانات ممكن تكون رقمية (Numeric) أو حرفية (Alphabetic) أو رقمية حرفية .(Alphanumeric)

الملفات ممكن تكون حرة الشكل (Free-Form) مثل الملفات النصية، أو منسّقة بصرامة .(Formatted Rigidly)

الملف عبارة عن تسلسل من Bits أو Bytes أو Lines أو Records، ومعناها يحدده منشئها.

Slide 14 - File Management (OS Activities)

- OS activities:
- The creation and deletion of files
- The creation and deletion of directories
- The support of primitives for manipulating files and directories
- The mapping of files onto secondary storage

- The backup of files on stable (non-volatile) storage media.

مهام نظام التشغيل في إدارة الملفات:

- إنشاء وحذف الملفات (Creation and Deletion of Files).
- إنشاء وحذف المجلدات (Directories).
- دعم العمليات الأساسية (Primitives) للتعامل مع الملفات والمجلدات.
- ربط الملفات بالتخزين الثانوي (Mapping onto Secondary Storage).
- عمل نسخ احتياطية (Backup) للملفات على وسائط تخزين ثابتة غير متطايرة (Non-Volatile).

Slide 15 - I/O System Management

- One of the purposes of the OS is to hide the peculiarities of specific hardware devices from the user.
- Unix hide the peculiarities of I/O devices from the bulk of the OS itself by the I/O subsystem.

من أهداف نظام التشغيل إخفاء خصوصيات وتفاصيل أجهزة الـ Hardware المحددة (Peculiarities) عن المستخدم.

نظام Unix يخفي تفاصيل أجهزة الـ I/O عن معظم أجزاء الـ OS نفسه، من خلال الـ I/O Subsystem.

Slide 16 - I/O System Management (cont.)

- Only the device drivers knows the peculiarities of the specific device to which it is assigned.

فقط الـ Device Drivers بتعرف تفاصيل وخصوصيات الجهاز المخصصة له.

بمعنى: كل جهاز إله Driver خاص فيه، والباقي من النظام بتعامل مع الـ Driver بدل ما يتعامل مع الجهاز مباشرة.

Slide 17 - Secondary-Storage Management

- Most modern computer systems use disks as the principle on-line storage medium, for both programs and data.
- OS activities:
- Free-space management
- Storage allocation
- Disk scheduling

معظم أنظمة الحاسوب الحديثة بتستخدم الأقراص (Disks) كوسيط التخزين الأساسي المتصل (On-Line) للبرامج والبيانات.

مهام نظام التشغيل هنا:

- Free-Space Management: إدارة المساحات الفاضية على القرص.
- Storage Allocation: تخصيص مساحة التخزين للملفات.
- Disk Scheduling: جدولة طلبات الوصول للقرص (ترتيب تنفيذها).

Slide 18 - Networking

- A distributed system is a collection of processors that do not share memory, peripheral devices, or a clock, and the processors communicate with one another through various communication lines, such as high-speed buses or networks.
- The processors in a distributed system vary in size and function. They may include small microprocessors, workstations, minicomputers, and large, general-purpose computer systems.
- OS usually generalize network access as a form of file access, with the details of networking being contained in the network interface's device driver.

الـ Distributed System هو مجموعة معالجات لا تشترك بالذاكرة ولا بالأجهزة الطرفية ولا بالـ Clock، وتتواصل مع بعضها عبر خطوط اتصال مثل High-Speed Buses أو الشبكات.

المعالجات في الـ Distributed System بتختلف بالحجم والوظيفة: من Microprocessors صغيرة إلى Workstations وMinicomputers وأنظمة كبيرة عامة الأغراض.

أنظمة التشغيل عادةً بتعامل الوصول للشبكة كأنه شكل من أشكال الوصول للملفات (Network Access as File Access)، وتفاصيل الشبكة بتكون داخل الـ Device Driver الخاص بكرت الشبكة (Network Interface).

Slide 19 - Protection System

- Protection: is any mechanism for controlling the access of programs, processes, or users to the resources defined by a computer:
- Memory-addressing hardware ensures that a process can execute only within its own address space.
- The timer ensures that no process can gain control of the CPU without eventually relinquishing control.
- Device control registers are not accessible to users, so that the integrity of the various peripheral devices is protected.

الـ Protection هي أي آلية للتحكم بوصول البرامج والـ Processes والمستخدمين إلى الموارد المعرّفة في الحاسوب.

مثل:

- Memory-Addressing Hardware: بضمن إن الـ Process تنفّذ فقط ضمن نطاق عناوينها (Address Space) الخاص فيها.
- Timer: بضمن إنه ما في Process بتقدر تستحوذ على الـ CPU للأبد؛ لازم ترجع التحكم في النهاية.
- Device Control Registers: ما بتكون متاحة للمستخدمين، عشان تنحفظ سلامة الأجهزة الطرفية (Integrity).

Slide 20 - Command-Interpreter System

- Command interpreter: the interface between the user and the OS.
- Many commands are given to the OS by control statements.
- A program that reads and interprets control statements is sometimes called control-card interpreter or the command-line interpreter, and is often known as the shell.

الـ Command Interpreter هو الواجهة بين المستخدم ونظام التشغيل.

كثير من الأوامر بتنعطى للـ OS بواسطة عبارات تحكم (Control Statements).

البرنامج اللي بقرأ عبارات التحكم وبفسرها بيُسمى أحيانًا Control-Card Interpreter أو Command-Line Interpreter، وغالبًا معروف باسم الـ Shell.

Slide 21 - Command-Interpreter System (cont.)

- Operating systems are frequently differentiate in the area of the shell, with a user-friendly command interpreter making the system more agreeable to some users.
- (e.g. mouse-based window and menu-based system)

أنظمة التشغيل كثيرًا بتختلف عن بعضها في منطقة الـ Shell.

وجود Command Interpreter سهل الاستخدام (User-Friendly) بخلي النظام أكثر قبولًا عند بعض المستخدمين.

مثال: أنظمة النوافذ المعتمدة على الماوس (Mouse-Based Window) والقوائم (Menu-Based).

Slide 22 - Command-Interpreter System (cont.)

- Command statements deal with:
- Process creation and managemen
- I/O handling
- Secondary-storage management
- Main-memory management
- File-system access
- Protection
- Networking

أوامر الـ Shell بتتعامل مع:

- إنشاء وإدارة الـ Processes.
- معالجة الـ I/O.
- إدارة التخزين الثانوي.
- إدارة الذاكرة الرئيسية.
- الوصول إلى نظام الملفات (File-System Access).
- الحماية (Protection).
- الشبكات (Networking).

Slide 23 - Operating System Services

- Operating Systems provides a number of services:
- At the lowest level, system calls allow a running program to make requests from the OS directly.
- At the higher level, the command interpreter or shell provides a mechanism for a user to issue a request without writing a program.
- System programs are provided to satisfy many common user requests.

نظام التشغيل بقدم عدة خدمات على مستويات مختلفة:

- أدنى مستوى: الـ System Calls بتسمح لبرنامج قيد التنفيذ يطلب من الـ OS مباشرة.
- مستوى أعلى: الـ Command Interpreter أو الـ Shell بوفر للمستخدم طريقة لإصدار طلب بدون ما يكتب برنامج.
- الـ System Programs موفّرة لتلبية كثير من طلبات المستخدمين الشائعة.

Slide 24 - Operating System Services (cont.)

- The types of requests may vary according to the level of the request:
- The system call level must provide the basic functions, such as process control and file and device management.
- Higher level requests, satisfied by the command interpreter or system programs, are translated into a sequence of system calls.

نوع الطلبات بيختلف حسب مستوى الطلب:

- على مستوى الـ System Call لازم تتوفر الوظائف الأساسية، مثل Process Control وإدارة الملفات والأجهزة.
- الطلبات ذات المستوى الأعلى، اللي بلبيها الـ Command Interpreter أو الـ System Programs، بتتحول إلى تسلسل من الـ System Calls.

Slide 25 - Operating System Services (cont.)

- Operating system services are:
- Program execution – system capability to load a program into memory and to run it.
- I/O operations – since user programs cannot execute I/O operations directly, the operating system must provide some means to perform I/O.
- File-system manipulation – program capability to read, write, create, and delete files.
- Communications – exchange of information between processes executing either on the same computer or on different systems tied together by a network. Implemented via shared memory or message passing.
- Error detection – ensure correct computing by detecting errors in the CPU and memory hardware, in I/O devices, or in user programs.

خدمات نظام التشغيل الخمس:

- Program Execution: قدرة النظام على تحميل برنامج للذاكرة وتشغيله.
- I/O Operations: برامج المستخدم ما بتقدر تنفّذ عمليات I/O مباشرة (لأنها Privileged Instructions)، فلازم الـ OS يوفر وسيلة لتنفيذها.
- File-System Manipulation: قدرة البرامج على قراءة وكتابة وإنشاء وحذف الملفات.
- Communications: تبادل المعلومات بين Processes على نفس الجهاز أو على أجهزة مختلفة مربوطة بشبكة، وبتتنفذ عبر Shared Memory أو Message Passing.
- Error Detection: ضمان صحة الحسابات باكتشاف الأخطاء في الـ CPU والذاكرة وأجهزة الـ I/O وبرامج المستخدم.

Slide 26 - Additional Operating System Functions

- Additional functions exist not for helping the user, but rather for ensuring efficient system operations.
- Resource allocation – allocating resources to multiple users or multiple jobs running at the same time.
- Accounting – keep track of and record which users use how much and what kinds of computer resources for account billing or for accumulating usage statistics.
- Protection – ensuring that all access to system resources is controlled.

في وظائف إضافية مش هدفها مساعدة المستخدم مباشرة، وإنما ضمان كفاءة تشغيل النظام:

- Resource Allocation: توزيع الموارد على عدة مستخدمين أو عدة Jobs تشتغل بنفس الوقت.
- Accounting: تتبّع وتسجيل مين يستخدم كم ونوع أي موارد من الحاسوب، لأغراض الفوترة (Billing) أو جمع إحصائيات الاستخدام.
- Protection: ضمان إن كل وصول لموارد النظام يكون مضبوط وخاضع للتحكم.

Slide 27 - Structure Development

- Once the system services are defined, the structure of the operating system can be developed.

بعد ما تتحدد خدمات النظام، بنقدر نطوّر هيكل (Structure) نظام التشغيل.

Slide 28 - System Calls

- System calls provide the interface between a running program (process) and the operating system.
- Generally available as assembly-language instructions.
- Languages defined to replace assembly language for systems programming allow system calls to be made directly (e.g., C, C++)
- Three general methods are used to pass parameters between a running program and the operating system.
- Pass parameters in registers.
- Store the parameters in a table in memory, and the table address is passed as a parameter in a register.
- Push (store) the parameters onto the stack by the program, and pop off the stack by operating system.

الـ System Calls هي الواجهة (Interface) بين البرنامج قيد التنفيذ (Process) ونظام التشغيل.

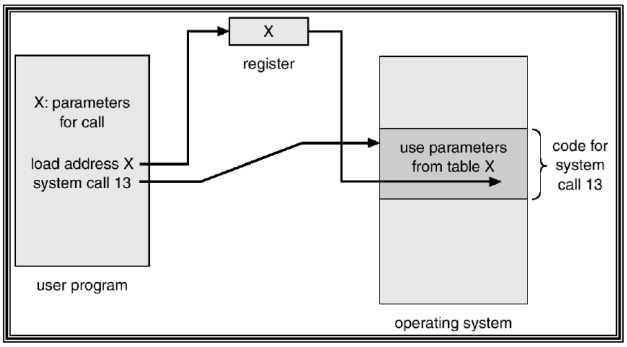
- عادةً بتكون متاحة كتعليمات بلغة التجميع (Assembly-Language Instructions).
- لغات البرمجة اللي وُجدت لتحل محل الـ Assembly في برمجة الأنظمة (مثل C و C++) بتسمح باستدعاء الـ System Calls مباشرة.

في ثلاث طرق عامة لتمرير المعاملات (Parameters) بين البرنامج والـ OS:

- تمرير المعاملات في الـ Registers.
- تخزين المعاملات في جدول (Table) بالذاكرة، ويُمرَّر عنوان الجدول كـ Parameter في Register.
- دفع (Push) المعاملات على الـ Stack من طرف البرنامج، ثم يسحبها نظام التشغيل.

Slide 29 - Passing of Parameters As A Table

الرسم يوضح الطريقة الثانية لتمرير الـparameters (Table):



- داخل البرنامج، المعاملات مخزّنة في جدول اسمه (X: parameters for call).
- البرنامج يحمّل عنوان الجدول X في Register.
- بعدها يطلب الخدمة رقم 13 من نظام التشغيل.
- داخل الـ OS، الكود الخاص بالـ System Call رقم 13 يقرأ محتوى الـ Register، ويستخدم المعاملات الموجودة في الجدول X.

الفكرة الأساسية: بدل ما نمرر كل المعاملات بالـ Registers، نمرر عنوان جدول واحد يحتويها كلها.

Slide 30 - Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications

الـ System Calls بتنقسم إلى خمسة أنواع:

- Process Control: التحكم بالـ Processes.
- File Management: إدارة الملفات.
- Device Management: إدارة الأجهزة.
- Information Maintenance: صيانة المعلومات.
- Communications: الاتصال.

Slide 31 - Process Control

- End, abort
- Load, execute
- Create process, terminate process
- Get process attributes, set process attributes (e.g. Job's priority, its maximum allowable execution time)
- Wait for time
- Wait event, signal event
- Allocate and free memory

أمثلة على الـ System Calls الخاصة بالتحكم بالـ Processes:

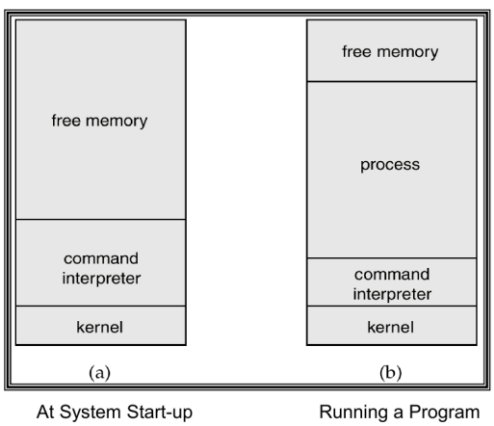
- End, Abort: إنهاء البرنامج بشكل طبيعي أو إجهاضه عند خطأ.
- Load, Execute: تحميل برنامج وتنفيذه.
- Create Process, Terminate Process: إنشاء Process وإنهاؤها.
- Get/Set Process Attributes: قراءة أو تعديل خصائص الـ Process، مثل أولوية الـ Job (Priority) وأقصى وقت تنفيذ مسموح له.
- Wait for Time: الانتظار لمدة زمنية محددة.
- Wait Event, Signal Event: الانتظار لحدث معيّن، أو الإشارة إلى حدوثه.
- Allocate and Free Memory: حجز الذاكرة وتحريرها.

Slide 32 - MS-DOS Execution

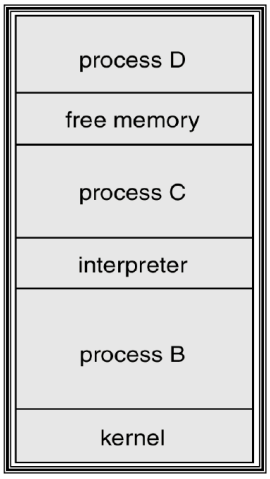
الرسم يبيّن ذاكرة MS-DOS في حالتين:

- (a) عند بدء تشغيل النظام (At System Start-up): الذاكرة فيها Kernel وCommand Interpreter، والباقي Free Memory.
- (b) أثناء تشغيل برنامج (Running a Program): بتظهر Process محمّلة في جزء من الذاكرة الفاضية، وتبقى المساحة المتبقية Free Memory.

لاحظ في الرسم إن مساحة الـ Command Interpreter بالحالة (b) أصغر منها بالحالة (a)



Slide 33 - UNIX Running Multiple Programs



الرسم يوضح ذاكرة UNIX وفيها أكثر من Process بنفس الوقت.

UNIX بقدر يشغّل عدة برامج مع بعض (Multiple Programs)، والـ Interpreter نفسه موجود بالذاكرة مع هالبرامج.

الفكرة الأساسية: الفرق عن MS-DOS: في UNIX عدة Processes بالذاكرة بنفس الوقت، بينما MS-DOS بيظهر فيه Process واحدة.

Slide 34 - File manipulation

- Create file, delete file
- Open, close
- Read, write, reposition
- Get file attributes, set file attributes (e.g. File name, file type, protection code, etc).

أمثلة على الـ System Calls الخاصة بالملفات:

- Create File, Delete File: إنشاء ملف وحذفه.
- Open, Close: فتح الملف وإغلاقه.
- Read, Write, Reposition: القراءة والكتابة وتحريك مؤشر الموقع داخل الملف.
- Get/Set File Attributes: قراءة أو تعديل خصائص الملف، مثل اسم الملف ونوعه وكود الحماية (Protection Code).

Slide 35 - Device manipulation

- Request device, release device
- Read, write, reposition
- Get device attributes, set device attributes
- Logically attach or detach devices

أمثلة على الـ System Calls الخاصة بالأجهزة:

- Request Device, Release Device: طلب جهاز وتحريره بعد الانتهاء.
- Read, Write, Reposition: القراءة والكتابة وإعادة التموضع (نفس فكرة الملفات).
- Get/Set Device Attributes: قراءة أو تعديل خصائص الجهاز.
- Logically Attach or Detach Devices: ربط الجهاز أو فصله منطقيًا.

Slide 36 - Information maintenance

- Get time or date, set time or date
- Get system data, set system data (e.g. Number of current users, version number of OS, the amount of free memory or disk space, etc).
- Get process, file, or device attributes
- Set process, file, or device attributes

أمثلة على الـ System Calls الخاصة بصيانة المعلومات:

- Get/Set Time or Date: قراءة أو ضبط الوقت والتاريخ.
- Get/Set System Data: قراءة أو ضبط بيانات النظام، مثل عدد المستخدمين الحاليين، رقم إصدار الـ OS، ومقدار الذاكرة أو مساحة القرص الفاضية.
- Get Process, File, or Device Attributes: قراءة خصائص Process أو ملف أو جهاز.
- Set Process, File, or Device Attributes: ضبط خصائصها.

Slide 37 - Communications

- Create, delete communication connection
- Send, receive messages
- Transfer status information
- Attach or detach remote devices

أمثلة على الـ System Calls الخاصة بالاتصال:

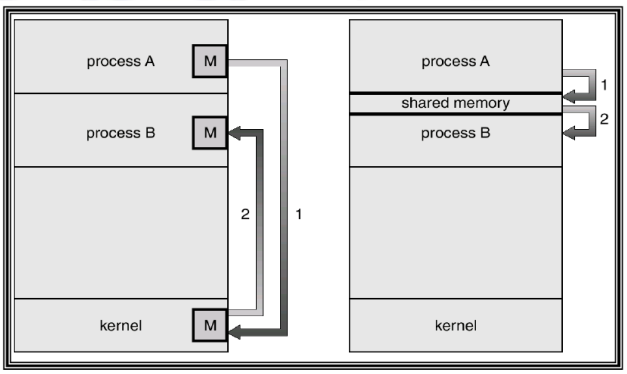
- Create, Delete Communication Connection: إنشاء وحذف اتصال.
- Send, Receive Messages: إرسال واستقبال الرسائل.
- Transfer Status Information: نقل معلومات الحالة.
- Attach or Detach Remote Devices: ربط أو فصل أجهزة بعيدة (Remote).

Slide 38 - Communication Models

- Communication may take place using either message passing or shared memory.

الاتصال بين الـ Processes ممكن يتم بطريقتين: Message Passing أو Shared Memory.

- **Msg Passing:** الرسالة M تنتقل من Process A إلى الـ Kernel ، ثم من الـ Kernel إلى Process B. يعني الرسالة بتمر عبر نظام التشغيل.
- **Shared Memory:** في منطقة ذاكرة مشتركة بين Process A و Process B؛ كل وحدة بتتعامل معها مباشرة
- **الفكرة الأساسية:** في الأولى الـ OS هو الوسيط، وفي الثانية الـ Processes نفسها بتتشارك منطقة من الذاكرة.



Slide 39 - Message-passing model

- Information is exchanged through an interprocess-communication facility provided by the OS.
- Before communication can take place, a connection must be opened.
- The name of the other communicator must be known.
- Each computer in a network has a host name (hostid).
- Each process has a process name (processid).
-

يتم تبادل المعلومات من خلال تسهيلات الاتصال بين العمليات (Interprocess-Communication Facility) اللي بوفرها نظام التشغيل.

قبل ما يبدأ الاتصال لازم تنفتح Connection.

لازم يكون اسم الطرف الثاني معروف:

- كل جهاز حاسوب في الشبكة إله Host Name (hosted).
- كل Process إلها Process Name (processed) .

Slide 40 - Shared-memory model

- Processes use map memory system calls to gain access to regions of memory owned by other processes.
- Requires that two or more processes agree to remove the restriction of preventing one process from accessing another process' memory.
- They may exchange information by reading and writing data in the shared areas.
- The processes are also responsible for ensuring that they are not writing to the same location simultaneously.

الـ Processes بتستخدم System Calls خاصة (Map Memory) للوصول لمناطق ذاكرة مملوكة لـ Processes ثانية.

هاد بتطلب إن أكثر من process يتفقوا على إزالة القيد اللي بمنع أي Process من الوصول لذاكرة Process ثانية بيتبادلوا المعلومات بقراءة وكتابة البيانات بالمناطق المشتركة (Shared Areas).

والـ Processes نفسها هي المسؤولة عن التأكد إنها ما تكتب على نفس الموقع بنفس اللحظة (Simultaneously).

الفكرة الأساسية: الحرية أكبر بالـ Shared Memory، لكن المسؤولية على الـ Processes نفسها مش على الـ OS.

Slide 41 - System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
- File manipulation
- Status information
- File modification
- Programming language support
- Program loading and execution
- Communications
- Application programs
- Most users' view of the operating system is defined by system programs, not the actual system calls.

الـ System Programs بتوفر بيئة مريحة (Convenient Environment) لتطوير البرامج وتنفيذها، وبتنقسم إلى:

- File Manipulation: التعامل مع الملفات.
- Status Information: معلومات حالة النظام.
- File Modification: تعديل الملفات.
- Programming Language Support: دعم لغات البرمجة.
- Program Loading and Execution: تحميل البرامج وتنفيذها.
- Communications: الاتصال.
- Application Programs: برامج التطبيقات.

تتكون فكرة معظم المستخدمين لنظام التشغيل من خلال الـ System Programs، مش من خلال الـ System Calls الفعلية.

الفكرة الأساسية: المستخدم بتعامل مع البرامج (مثل المحرر والمترجم)، وهي بدورها بتستدعي الـ System Calls.

Slide 42 - System Structure

- A system as large and complex as a modern OS must be engineered properly and be modified easily.
- Since the OS is large, modularity is important. Designing a system as a sequence of layers or using microkernel is considered a good technique.

نظام كبير ومعقد مثل نظام التشغيل الحديث لازم يُصمَّم هندسيًا بشكل صحيح ويكون سهل التعديل.

بما إن الـ OS كبير، فالـ) Modularityالتقسيم لوحدات) مهمة.

تصميم النظام كتسلسل من الطبقات (Layers) أو باستخدام Microkernel يُعتبر تقنية جيدة.

الفكرة الأساسية: الأقسام القادمة بتقارن بين طرق بناء الهيكل: Simple Structure، ثم Layered Approach، ثم Microkernel، ثم Virtual Machines.

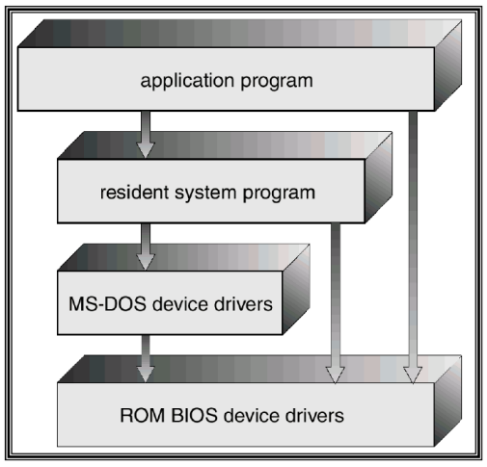
Slide 43 - Simple Structure

- MS-DOS System Structure
- MS-DOS – written to provide the most functionality in the least space
- not divided into modules
- Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated

الـ Simple Structure مثل MS-DOS.

- MS-DOS انكتب ليعطي أكبر قدر من الوظائف بأقل مساحة. (Most Functionality in the Least Space)
- غير مقسّم إلى وحدات. (Not Divided into Modules)
- رغم إن فيه بعض الهيكلة، إلا إن واجهاته ومستويات امكانيات غير مفصولة بشكل جيد. (Not Well Separated)

Slide 44 - MS-DOS Layer Structure



الرسم يوضح مستويات MS-DOS من الأعلى للأسفل

الأسهم بالرسم بتبيّن إن الـ Application Program والـ Resident System Program بيقدروا يوصلوا مباشرة إلى الـ ROM BIOS Device Drivers، متجاوزين المستويات اللي بينهم.

هاد معنى "غير مفصولة بشكل جيد": أي طبقة بتقدر توصل للأسفل مباشرة، فما في حماية حقيقية بين المستويات.

Slide 45 - UNIX System Structure

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts.
- Systems programs
- The kernel
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level.

نظام UNIX الأصلي كان محدود الهيكلة (Limited Structuring) بسبب محدودية إمكانيات الـ Hardware.

نظام UNIX بتكوّن من جزئين قابلين للفصل:

- System Programs.
- الـ Kernel: بتضم كل شي تحت واجهة الـ System-Call وفوق الـ Physical Hardware.

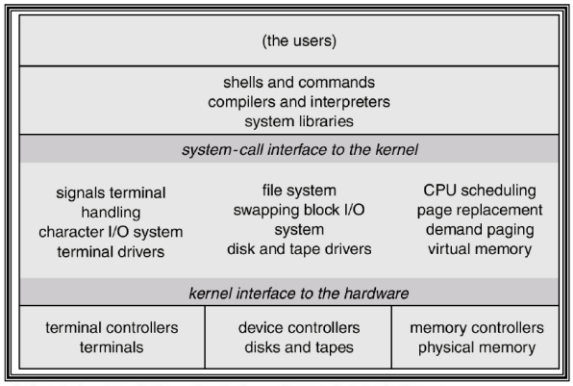
الـ Kernel بوفر نظام الملفات (File System) وجدولة الـ CPU وإدارة الذاكرة ووظائف نظام التشغيل الأخرى، أي عدد كبير من الوظائف في مستوى واحد.

الفكرة الأساسية: نقطة الضعف: كل هالوظائف الكثيرة مجمّعة في مستوى واحد (Kernel كبير).

Slide 46 - UNIX System Structure (Diagram)

الرسم يبيّن طبقات نظام UNIX من الأعلى للأسفل:

كل ما بين واجهتين (System-Call Interface و Kernel Interface to the Hardware) هو الـ Kernel.



Slide 47 - Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers.

في الـ Layered Approach بنقسم نظام التشغيل إلى عدد من الطبقات (Layers أو Levels)، كل طبقة مبنية فوق الطبقات اللي تحتها.

الطبقة السفلية (Layer 0) هي الـ Hardware، والطبقة العليا (Layer N) هي واجهة المستخدم (User Interface).

بالـ Modularity بتنختار الطبقات بحيث كل طبقة تستخدم وظائف (Operations) وخدمات الطبقات الأدنى منها فقط.

الفكرة الأساسية: كل طبقة بتعتمد على اللي تحتها فقط، ولا بتعتمد أبدًا على اللي فوقها.

Slide 48 - Layered Approach (Advantages)

- This approach simplifies debugging and system verification.
- The design and implementation of the system are simplified when the system is broken down into layers.
- Each layer is implemented with only those operations provided by lower-level layers.
- A layer does not need to know how these operations are implemented; it needs to know only what these operations do.

مزايا الـ Layered Approach:

- بسّط عملية اكتشاف الأخطاء (Debugging) والتحقق من النظام (System Verification).
- بسّط تصميم النظام وتنفيذه عند تقسيمه إلى طبقات.
- كل طبقة بتتبني باستخدام العمليات اللي توفرها الطبقات الأدنى فقط.
- الطبقة ما بتحتاج تعرف كيف نُفّذت هالعمليات (How)، بتحتاج تعرف فقط شو بتعمل (What).
-

Slide 49 - Layered Approach (Difficulties)

- The major difficulty with the layered approach involves the careful definition of the layers, because a layer can use only those layers below it (e.g. The device driver for the disk space used by virtual-memory algorithms must be at a level lower than that of the memory management routines).
- Layered implementations tend to be less efficient (overhead).

الصعوبة الرئيسية في الـ Layered Approach هي التعريف الدقيق للطبقات، لأن الطبقة بتقدر تستخدم الطبقات اللي تحتها فقط.

مثال: الـ Device Driver الخاص بمساحة القرص المستخدمة من خوارزميات الذاكرة الافتراضية (Virtual Memory) لازم يكون بمستوى أدنى من مستوى روتينات إدارة الذاكرة (Memory Management Routines).

الصعوبة الثانية: التنفيذات الطبقية تميل لتكون أقل كفاءة (Less Efficient) بسبب الـ Overhead، لأن الطلب لازم يمر عبر عدة طبقات.

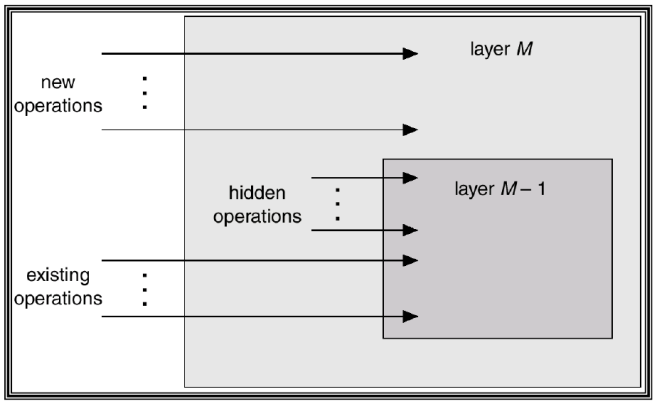
الترتيب الصحيح للطبقات صعب، والمرور عبر الطبقات كلها بكلّف وقت إضافي.

Slide 50 - An Operating System Layer

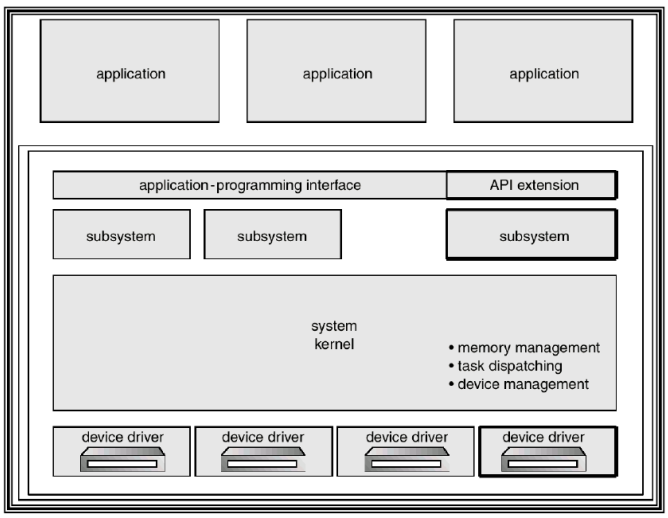
الرسم يوضح طبقة من طبقات نظام التشغيل (Layer M) وتحتها الطبقة Layer M – 1

- New Operations: العمليات الجديدة اللي بتضيفها Layer M وبتقدمها للطبقات الأعلى.
- Existing Operations: عمليات موجودة أصلًا في Layer M – 1 وبتضل متاحة من خلال Layer M.
- Hidden Operations: عمليات تستخدمها Layer M داخليًا مع Layer M – 1 ولا تظهر للمستوى الأعلى.

كل طبقة بتقدم لمن فوقها مجموعة عمليات، وبتخفي عنه التفاصيل الداخلية للطبقة التي تحتها.



Slide 51 - OS/2 Layer Structure



OS/2 مثال على تطبيق الـ Layered Approach: التطبيقات فوق الـ API، وتحتها الـ Kernel، وتحتها الـ Drivers.

Slide 52 - Microkernel System Structure

- Moves as much from the kernel into “user” space.
- Communication takes place between user modules using message passing.
- In general, microkernel typically provide minimal process and memory management, in addition to a communication facility.

الـ Microkernel بنقل أكبر قدر ممكن من وظائف الـ Kernel إلى (User Space).

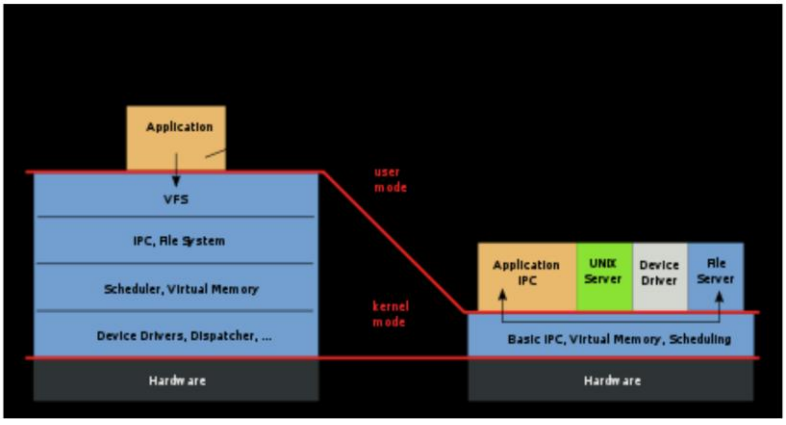
الاتصال بين وحدات المستخدم (User Modules) بيتم باستخدام Message Passing.

عادةً الـ Microkernel بوفر الحد الأدنى من إدارة العمليات والذاكرة (Minimal Process and Memory Management)، بالإضافة إلى تسهيلات الاتصال (Communication Facility).

Slide 53 - Microkernel (Diagram)

الرسم بيقارن بين Kernel تقليدي (يسار) و Microkernel (يمين):

في الـ Microkernel خرجت الخدمات (File Server وDevice Driver وغيرها) من الـ Kernel لتصير Servers في User Mode.



Slide 54 - Microkernel (Definition)

- A microkernel is the near-minimum amount of software that can provide the mechanisms needed to implement an operating system. These mechanisms include low-level address space management, thread management, and inter-process communication (IPC).
- As an operating system design approach, microkernels permit typical operating system services, such as device drivers, protocol stacks, file systems and user interface code, to run in user space.

الـ Microkernel هو الحد الأدنى تقريبًا من البرمجيات القادر على توفير الآليات (Mechanisms) اللازمة لتنفيذ نظام تشغيل.

من هالآليات: إدارة فضاء العناوين على المستوى المنخفض (Low-Level Address Space Management)، وإدارة الـ Threads، والاتصال بين العمليات (IPC).

كأسلوب تصميم، الـ Microkernel بسمح لخدمات نظام التشغيل المعتادة، مثل Device Drivers وProtocol Stacks وFile Systems وكود واجهة المستخدم، بالتشغيل في User Space.

Slide 55 - Microkernels (Benefits)

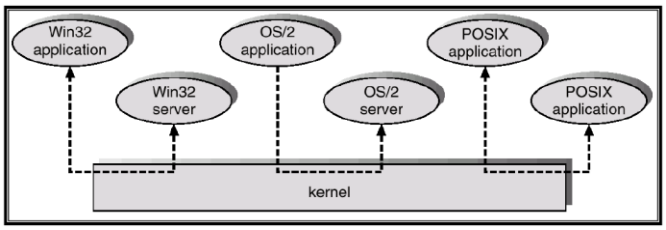
- Benefits:
- Easier to extend a microkernel:
    - all new services are added to user space and consequently do not require modifications of the kernel.
    - When the kernel does have to be modified, the changes tend to be fewer.
- Easier to port the operating system to new architectures
- More reliable (less code is running in kernel mode)
- More secure

فوائد الـ Microkernel:

- أسهل بالتوسعة (Easier to Extend): كل الخدمات الجديدة يتم اضافتها في User Space، فما بتحتاج تعديل الـ Kernel. وحتى لما يتطلب تعديل الـ Kernel، التغييرات بتكون أقل.
- أسهل بالنقل (Easier to Port) لمعماريات Hardware جديدة.
- موثوق اكثر (More Reliable): لأن كود أقل بشتغل في Kernel Mode.
- أكثر أمانًا (More Secure).

كود أقل داخل الـ Kernel= أخطاء أقل بتأثر على النظام كله.

Slide 56 - Windows NT Client-Server Structure



الرسم بيبيّن هيكل Windows NT بأسلوب Client-Server:

- في الأعلى ثلاث أنواع تطبيقات: Win32 Application وOS/2 Application وPOSIX Application.
- لكل نوع في Server خاص فيه: Win32 Server وOS/2 Server وPOSIX Server.
- الـ Kernel في الأسفل، والأسهم المتقطعة بتبيّن إن الاتصال بين التطبيق والـ Server بمر عبر الـ Kernel.

التطبيق (Client) يطلب خدمة من الـ Server المناسب لنوعه، والـ Kernel وسيط الاتصال بينهم

Slide 57 - Virtual Machines

- A virtual machine takes the layered approach to its logical conclusion. It treats hardware and the operating system kernel as though they were all hardware. Other Operating systems may even be loaded on top of this virtual machine.
- A virtual machine provides an interface identical to the underlying bare hardware.
- The operating system creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.

الـ Virtual Machine بتاخد الـ Layered Approach لنهايتها المنطقية: بتتعامل مع الـ Hardware وال kernel وكأنهم كلهم Hardware.

وحتى ممكن تحميل أنظمة تشغيل أخرى فوق ال Virtual Machine.

الـ Virtual Machine بتوفر واجهة مطابقة تمامًا للـ Bare Hardware اللي تحتها

نظام التشغيل بيخلق عدة Processes وهميه، كل وحدة بتنفذ على معالج خاص فيها وبذاكرة (افتراضية) خاصة فيها.

كل مستخدم بحس إنه عنده حاسوب كامل لحاله، مع إن الجهاز الفيزيائي واحد.

Slide 58 - Virtual Machines (cont.)

- The resources of the physical computer are shared to create the virtual machines.
- CPU scheduling can create the appearance that users have their own processor.
- Spooling and a file system can provide virtual card readers and virtual line printers.
- A normal user time-sharing terminal serves as the virtual machine operator's console.

موارد الحاسوب بتتعاون لإنشاء الـ Virtual Machines:

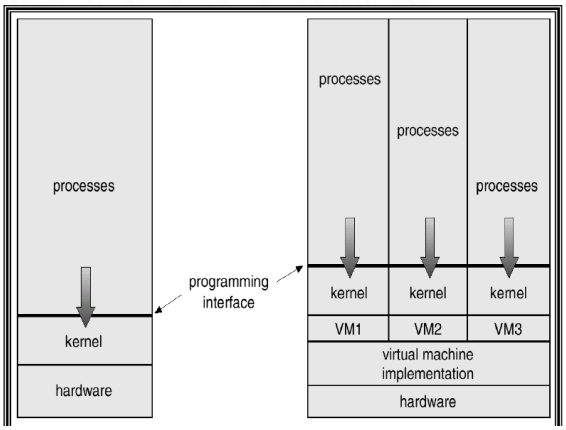
- الـ CPU Scheduling بخلق مظهر إن كل مستخدم إله معالج خاص فيه.
- الـ Spooling ونظام الملفات (File System) بقدروا يوفروا Virtual Card Readers وVirtual Line Printers.
- الـ Terminal العادي بنظام ال (Time-Sharing) بشتغل كأنه Console خاص بمشغّل الـ Virtual Machine.

Slide 59 - System Models

الرسم بيقارن بين نموذجين:

- **Non-Virtual Machine(يسار):** Processes فوق Kernel واحد، وهو فوق الـ Hardware مباشرة.
- **Virtual Machine(يمين):** عدة Processes، وكل مجموعة إلها Kernel خاص فيها (لكل من VM1 وVM2 وVM3).

في الـ Virtual Machine بتحمّل عدة أنظمة تشغيل (كل وحدة بـ Kernel خاص) على نفس الـ Hardware.



Slide 60 - Advantages/Disadvantages of Virtual Machines

- The virtual-machine concept provides complete protection of system resources since each virtual machine is isolated from all other virtual machines. This isolation, however, permits no direct sharing of resources.
- A virtual-machine system is a perfect vehicle for operating-systems research and development. System development is done on the virtual machine, instead of on a physical machine and so does not disrupt normal system operation.
- The virtual machine concept is difficult to implement due to the effort required to provide an exact duplicate to the underlying machine.

المزايا:

- حماية كاملة لموارد النظام (Complete Protection)، لأن كل Virtual Machine معزولة (Isolated) عن باقي الـ VMs.
- بيئة مثالية لأبحاث وتطوير أنظمة التشغيل: التطوير بتم على الـ Virtual Machine بدل الجهاز الفيزيائي، فما بعطّل عمل النظام الطبيعي.

العيوب:

- هالعزل ما بسمح بمشاركة مباشرة للموارد (No Direct Sharing of Resources).
- المفهوم صعب التنفيذ، بسبب الجهد المطلوب لتوفير نسخة طبق الأصل من الجهاز الذي تحتها (Exact Duplicate).

Slide 61 - Java Virtual Machine

- Compiled Java programs are platform-neutral bytecodes executed by a Java Virtual Machine (JVM).
- JVM consists of
- class loader
- class verifier
- runtime interpreter
- Just-In-Time (JIT) compilers increase performance

برامج Java بعد الـ Compile بتصير Bytecodes محايدة للمنصة (Platform-Neutral)، بتنفّذها الـ Java Virtual Machine (JVM).

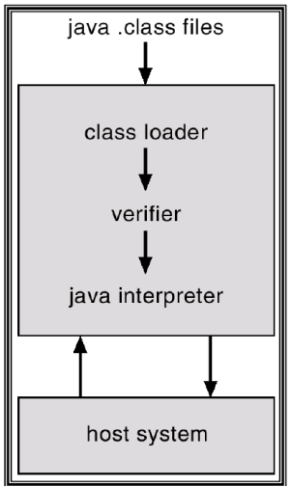
الـ JVM بتتكون من:

- Class Loader: بحمّل ملفات الـ Classes.
- Class Verifier: بتحقق من صحتها وسلامتها.
- Runtime Interpreter: بنفّذ الـ Bytecodes.

مترجمات Just-In-Time (JIT) بتزيد الأداء.

الفكرة الأساسية: نفس برنامج Java بشتغل على أي نظام فيه JVM، لأنه بتوجه للـ JVM مش للـ Hardware مباشرة.

Slide 62 - Java Virtual Machine (Diagram)



الرسم يوضح مراحل عمل الـ JVM بالترتيب:

- ملفات java .class تدخل أولًا على الـ Class Loader.
- بعدها الـ Verifier يتحقق منها.
- بعدها الـ Java Interpreter ينفّذها.
- الـ Java Interpreter بتواصل مع الـ Host System بالاتجاهين .

Slide 63 - System Design Goals

- User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast.
- System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient.

عند تصميم نظام التشغيل في نوعين من الأهداف:

- User Goals(أهداف المستخدم): النظام لازم يكون مريح الاستخدام (Convenient)، سهل التعلم (Easy to Learn)، موثوق (Reliable)، آمن (Safe)، وسريع .(Fast)
- System Goals(أهداف النظام): لازم يكون سهل التصميم والتنفيذ والصيانة، ومرن (Flexible)، وموثوق، وخالٍ من الأخطاء (Error-Free)، وكفء .(Efficient)

Slide 64 - Mechanisms and Policies

- Mechanisms determine how to do something, policies decide what will be done.
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later.

الـ Mechanisms بتحدد كيف (How) نعمل الشي، أما الـ Policies فبتقرر شو (What) اللي رح ينعمل.

فصل الـ Policy عن الـ Mechanism مبدأ مهم جدًا، لأنه بعطي أقصى مرونة لو احتجنا نغيّر قرارات الـ Policy لاحقًا.

Slide 65 - System Implementation

- Traditionally written in assembly language (systems implementation language), operating systems can now be written in higher-level languages.
- Code written in a high-level language:
- can be written faster.
- is more compact.
- is easier to understand and debug.
- An operating system is far easier to port (move to some other hardware) if it is written in a high-level language.

تقليديًا كانت أنظمة التشغيل تنكتب بلغة التجميع (Assembly Language)، وهلأ بتنكتب بلغات عالية المستوى (High-Level Languages).

الكود المكتوب بلغة عالية المستوى:

- بنكتب أسرع (Can Be Written Faster).
- أكثر اختصارًا (More Compact).
- أسهل بالفهم والتصحيح (Easier to Understand and Debug).

ونظام التشغيل بيكون أسهل بكثير بالنقل لـ Hardware آخر إذا انكتب بلغة عالية المستوى.

Slide 66 - System Generation (SYSGEN)

- Operating systems are designed to run on any of a class of machines; the system must be configured for each specific computer site.
- SYSGEN program obtains information concerning the specific configuration of the hardware system.
- Booting – starting a computer by loading the kernel.
- Bootstrap program – code stored in ROM that is able to locate the kernel, load it into memory, and start its execution.

أنظمة التشغيل بتنصمم لتشتغل على أي جهاز من فئة معينة من الأجهزة، لكن لازم يتم ضبط النظام (Configuration) لكل موقع حاسوب محدد.

برنامج الـ SYSGEN بجمع معلومات عن الإعداد المحدد لنظام الـ Hardware.

- Booting: تشغيل الحاسوب بتحميل الـ Kernel.
- Bootstrap Program: كود مخزّن في الـ ROM قادر على تحديد مكان الـ Kernel، وتحميله في الذاكرة، وبدء تنفيذه.

Chapter 3 – Comprehensive Review

Important Definitions

Term	Definition / Key Idea
Process	A program in execution; the unit of work in a system.
Program	A passive entity, such as the contents of a file stored on a disk.
Program Counter	Specifies the next instruction that an active process will execute.
File	A collection of related information defined by its creator.
Device Driver	The only component that knows the peculiarities of the specific device to which it is assigned.
Distributed System	A collection of processors that do not share memory, peripheral devices, or a clock, and communicate through communication lines.
Protection	Any mechanism for controlling the access of programs, processes, or users to the resources defined by a computer.
Command Interpreter (Shell)	The interface between the user and the OS; reads and interprets control statements.
System Call	The interface between a running program (process) and the operating system.
System Programs	Programs that provide a convenient environment for program development and execution.
Resource Allocation	Allocating resources to multiple users or multiple jobs running at the same time.
Accounting	Keeping track of and recording which users use how much and what kinds of computer resources.
Message Passing	Communication through an interprocess-communication facility provided by the OS.
Shared Memory	Communication through memory regions that processes agree to share (gained via map memory system calls).
Simple Structure	An OS that is not divided into well-separated modules or levels (e.g., MS-DOS).
Kernel (UNIX)	Everything below the system-call interface and above the physical hardware.
Layered Approach	The OS is divided into layers, each built on lower layers; layer 0 is the hardware and layer N is the user interface.
Microkernel	The near-minimum amount of software that provides the mechanisms needed to implement an OS; moves as much as possible from the kernel into user space.
Virtual Machine	Provides an interface identical to the underlying bare hardware, creating the illusion that each process has its own processor and memory.
Java Virtual Machine (JVM)	Executes platform-neutral Java bytecodes; consists of a class loader, class verifier, and runtime interpreter.
Just-In-Time (JIT) Compiler	A compiler that increases the performance of Java execution.
Mechanism	Determines how to do something.
Policy	Decides what will be done.
SYSGEN	A program that obtains information about the specific hardware configuration so that the OS can be configured for a specific computer site.
Booting	Starting a computer by loading the kernel.
Bootstrap Program	Code stored in ROM that locates the kernel, loads it into memory, and starts its execution.

System Components and Main OS Activities

Component	Main OS Activities
Process Management	Creation and deletion of processes; suspension and resumption; synchronization, communication and deadlock-handling mechanisms.
Main-Memory Management	Keep track of used memory and by whom; decide which processes to load; allocate and deallocate memory.
File Management	Creation and deletion of files and directories; file/directory primitives; mapping files onto secondary storage; backup.
I/O System Management	Hide device peculiarities from the user; only device drivers know the specific device.
Secondary-Storage Management	Free-space management; storage allocation; disk scheduling.
Networking	Generalize network access as a form of file access; details are inside the network interface's device driver.
Protection System	Control access of programs, processes, or users to resources (memory addressing, timer, device control registers).
Command-Interpreter System	Interface between user and OS (the shell); commands deal with processes, I/O, storage, memory, files, protection, networking.

Types of System Calls

Category	Examples
Process control	End/abort; load/execute; create/terminate process; get/set process attributes; wait for time; wait/signal event; allocate/free memory.
File management	Create/delete file; open/close; read/write/reposition; get/set file attributes.
Device management	Request/release device; read/write/reposition; get/set device attributes; logically attach/detach devices.
Information maintenance	Get/set time or date; get/set system data; get/set process, file, or device attributes.
Communications	Create/delete communication connection; send/receive messages; transfer status information; attach/detach remote devices.

Passing Parameters to the OS (System Calls)

Method	How It Works
In registers	The parameters are passed directly in registers.
In a table in memory	The parameters are stored in a table in memory, and the table address is passed as a parameter in a register.
On the stack	The program pushes (stores) the parameters onto the stack, and the OS pops them off the stack.

Comparison: Program vs. Process

Program	Process
Passive entity.	Active entity.
Contents of a file stored on a disk.	A program in execution.
Does not execute by itself.	Progresses sequentially; at most one instruction is executed on its behalf at any time.
Can be associated with several processes.	Two processes of the same program are separate execution sequences.
Static code and data.	The unit of work in a system, with a program counter for the next instruction.

Comparison: System Calls vs. Command Interpreter / System Programs

System Calls	Command Interpreter / System Programs
Lowest level of service.	Higher level of service.
A running program makes requests to the OS directly.	A user issues a request without writing a program.
Provide basic functions: process control, file and device management.	Requests are translated into a sequence of system calls.
Generally available as assembly-language instructions (also callable from C and C++).	The shell is the command interpreter; system programs satisfy common user requests.

Comparison: Message Passing vs. Shared Memory

Message Passing	Shared Memory
Information is exchanged through an IPC facility provided by the OS.	Processes read and write data in shared memory areas.
A connection must be opened before communication.	Map memory system calls are used to gain access to memory owned by other processes.
The name of the other communicator must be known (hostid, processid).	Processes must agree to remove the restriction that prevents access to each other's memory.
Messages pass through the kernel (see diagram).	Processes share a memory region directly (see diagram).
The OS handles the exchange.	The processes are responsible for not writing to the same location simultaneously.

Comparison: MS-DOS vs. UNIX (Structure)

MS-DOS	UNIX
Written to provide the most functionality in the least space.	Limited by hardware functionality; the original UNIX had limited structuring.
Not divided into modules.	Two separable parts: system programs and the kernel.
Interfaces and levels of functionality are not well separated.	The kernel provides a large number of functions for one level.
Application programs can access lower levels (even ROM BIOS drivers) directly.	Users and system programs reach the kernel through the system-call interface.

Comparison: MS-DOS Execution vs. UNIX Running Multiple Programs

MS-DOS Execution	UNIX Running Multiple Programs
One process is shown in memory (single program at a time).	Several processes are in memory at the same time.
Memory holds the kernel, command interpreter, process, and free memory.	Memory holds the kernel, interpreter, processes B, C and D, and free memory.
Command interpreter area is smaller while a program runs (as in the diagram).	The interpreter stays in memory alongside the running processes.

Comparison: Layered Approach vs. Microkernel

Layered Approach	Microkernel
The OS is divided into layers, each built on top of lower layers.	Moves as much as possible from the kernel into user space.

Layered Approach	Microkernel
Each layer uses only the operations of lower layers.	User modules communicate using message passing.
Simplifies debugging and system verification.	Easier to extend and easier to port to new architectures.
Careful definition of the layers is required.	More reliable and more secure (less code runs in kernel mode).
Less efficient because of overhead.	Extra communication cost between user modules (general knowledge, not on the slides).
Example on the slides: OS/2 layer structure.	Example on the slides: Windows NT client-server structure.

Comparison: Non-Virtual Machine vs. Virtual Machine

Non-Virtual Machine	Virtual Machine
Processes run on a single kernel over the hardware.	Each VM (VM1, VM2, VM3) has its own kernel and processes.
The kernel runs directly on the hardware.	VMs run on a virtual machine implementation layer above the hardware.
The programming interface is provided by the kernel.	Provides an interface identical to the underlying bare hardware.
Resources are shared normally among processes.	Complete protection through isolation, but no direct sharing of resources.

Comparison: Mechanism vs. Policy

Mechanism	Policy
Determines how to do something.	Decides what will be done.
Example (outside the slides): the timer that interrupts the CPU after a period.	Example (outside the slides): deciding how long the timer is set for each user.
Should stay fixed when policies change.	Can be changed later; separating it from the mechanism gives maximum flexibility.

Comparison: User Goals vs. System Goals

User Goals	System Goals
Viewpoint of the users.	Viewpoint of the designers and implementers.
Convenient to use, easy to learn.	Easy to design, implement, and maintain.
Reliable and safe.	Flexible, reliable, and error-free.
Fast.	Efficient.

Comparison: OS Services vs. Additional OS Functions

OS Services (help the user)	Additional OS Functions (ensure efficient operation)
Program execution	Resource allocation
I/O operations; file-system manipulation	Accounting
Communications (shared memory or message passing)	Protection
Error detection	—